Análisis de Algoritmos II

Análisis aproximado

Para determinar aproximadamente el tiempo de ejecución se:

- Identifican los distintos bloques, que requieren un tiempo de ejecución constante
- Se determina la frecuencia de ejecución

Se multiplican los tiempos de cada bloque por su frecuencia y se suman todos los resultados para el estimado final.

Ejemplos de análisis - 1

Encontrar el máximo de un vector

```
double max(double[] a) {
    double x = a[0];
    for(int i=1; i<a.length; i++)
        if (a[i]>x) x=a[i];
    return x;
}
```

~N

Ejemplos de análisis - 2

Encontrar parejas que sumen 0

```
int paresCero(int[] a) {
    int conteo=0;
    for(int i=0; i<a.length; i++)
        for(int j=0; j<a.length; j++)
            if (i!=j && a[i]+a[j]==0)
                conteo++;
    return conteo;
}
```

$\sim N^2$

Obtener todos los subconjuntos de un vector $\sim 2^N$

Análisis simplificado

- Se define una ***operación elemental*** como toda aquella operación cuyo tiempo está acotado por una constante.
- Observamos que generalmente el orden de crecimiento depende de las instrucciones del ciclo más interno, o alternativamente, la ***operación elemental*** de mayor frecuencia.
- El orden de crecimiento obtenido por el análisis aproximado *es* ***independiente de la implementación***.

Ejemplo
Análisis de ThreeSum

```
public static int count(int[] a) {
    int n = a.length;
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i+1; j < n; j++) {
            for (int k = j+1; k < n; k++) {
                if (a[i] + a[j] + a[k] == 0) {
                    count++;
                }
            }
        }
    }
    return count;
}
```

Algunas series de uso frecuente

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1}, \quad r \neq 1$$

Solución Threesum

$$
\begin{aligned}
T(n) &= \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \sum_{k=j+1}^{n-1} 1 \\
&= \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (n-j-1) \\
&= \sum_{i=0}^{n-1} \left(\sum_{j=i+1}^{n-1} n - \sum_{j=i+1}^{n-1} j - \sum_{j=i+1}^{n-1} 1 \right) \\
&= \sum_{i=0}^{n-1} \left(n(n-i-1) - \left[\frac{n(n-1)}{2} - \frac{i(i+1)}{2} \right] - (n-i-1) \right) \\
&= \sum_{i=0}^{n-1} \left((n-1)(n-i-1) - \frac{n(n-1)}{2} + \frac{i(i+1)}{2} \right) \\
&= (n-1) \sum_{i=0}^{n-1} (n-i-1) - \sum_{i=0}^{n-1} \frac{n(n-1)}{2} + \frac{1}{2} \sum_{i=0}^{n-1} (i^2+i) \\
&= (n-1) \cdot \frac{(n-1)n}{2} - n \cdot \frac{n(n-1)}{2} + \frac{1}{2} \left[\frac{(n-1)n(2n-1)}{6} + \frac{(n-1)n}{2} \right] \\
&= \frac{(n-1)^2 n}{2} - \frac{n^2(n-1)}{2} + \frac{(n-1)n(2n-1)}{12} + \frac{(n-1)n}{4} \\
&= \frac{n(n-1)}{12} \left[6(n-1) - 6n + (2n-1) + 3 \right] \\
&= \frac{n(n-1)}{12} \left[2n - 4 \right] \\
&= \frac{n(n-1)(n-2)}{6} \\
&= \binom{n}{3} \\
&\sim \frac{n^3}{6}
\end{aligned}
$$

Modelo de costo

Muchas veces interesa contabilizar cierto tipo de operación que tiene un impacto alto en el desempeño. Típicamente, operaciones de aritmético-lógicas o de acceso a la memoria.

Ejemplo: En ThreeSum

- Modelo de costo accesos al arreglo: 3 por iteración
- Frecuencia del condicional: $N^3/6$
- Número total de accesos: $3 \cdot (N^3/6) = N^3/2$

Ejemplo: Modelos de costo

Evaluar un polinomio, método por definición:

```
public static double evalPoly(double[] a, double x)
{
   double s = 0;
   for(int i=0; i<a.length; i++)
      s += a[i]*Math.pow(x,i);
   return s;
}
```

Modelos de costo:

- Accesos a memoria, a[i] : d+1
- Operaciones suma (double) : d+1
- Operaciones producto (double) : d+1+d(d-1)/2

Ejemplo: Modelos de costo

Evaluar un polinomio, método por regla de Horner:

```
public static double horner(double[] a, double x) {
   double s = a[a.length-1];
   for(int i=a.length-2; i>=0; i--)
      s = s*x + a[i];
   return s;
}
```

Modelos de costo:
- Accesos a memoria, a[i] : d+1
- Operaciones suma (double) : d
- Operaciones producto (double) : d

Metodología general de análisis

1. Se identifica el modelo de la entrada (Cuál es el tamaño N de la entrada).
2. Se define el modelo de costo para el ciclo más interno.
3. Se calcula la frecuencia de ejecución de las operaciones asociadas al modelo de costo en el ciclo más interno.

Notación Big-O

- Es una notación ampliamente utilizada para describir el comportamiento *asintótico de funciones*, es decir para valores grandes del tamaño de la entrada.
- Decimos que T(N) esta en el orden O(f(N)) si:

Para N suficientemente grandes,

existe una constante c>0 tal que

$$T(N) \leq c * f(N)$$

- En otras palabras T(N) está acotada superiormente por un factor constante de f(N) para todo N grande.

Jerarquía de conjuntos “Big-O”

$O(1)$	Constante
$O(\log n)$	Logaritmico
$O(n)$	Lineal
$O(n \log n)$	Linearitmetico
$O(n^2)$	Cuadratico
$\ldots$	
$O(n^k)$	Polinomico
$\ldots$	
$O(b^n)$	Exponencial

Experimentos de doblado de la entrada

- Aplicable en casos en los que los algoritmos crecen como funciones de potencia

$$N^b$$

1. Se mide experimentalmente duplicando el tamaño de la entrada en cada caso.
2. Si el cociente entre tiempos sucesivos es 2^b, entonces el algoritmo tiene un orden de crecimiento N^b.

Ejemplo

- Sea un algoritmo con $T(N) = a\,N^b\,(\lg N)^c$.
- La relación T(2N)/T(N) sería:

$$\begin{aligned}
\frac{T(2N)}{T(N)} &= \frac{a(2N)^b(\lg 2N)^c}{aN^b(\lg N)^c} \\
&= 2^b\left(\frac{\lg 2 + \lg n}{\lg N}\right)^c \\
&= 2^b\left(1 + \frac{\lg 2}{\lg N}\right)^c \\
&\approx 2^b
\end{aligned}$$

Ejemplo:
Prueba de doblado con ThreeSum

N	T(N)	T(2N) / T(N)
500	0.1	
1000	0.1	1.0
2000	0.6	6.0
4000	4.7	7.8
8000	36.3	7.7
16000	287.0	7.9

Observamos que el cociente T(2N)/T(N) tiende a $7.9=2^{2.98}$
Bastante cercano respecto al exponente esperado teóricamente (3)

Estimando tiempos para entradas mayores

Una vez los tiempos convergen, se puede estimar el tiempo para entradas mayores:

- Se duplica N y se multiplica por 2^b el tiempo

Limitantes de la aproximación de doblado de entrada

- Constantes grandes
- Loop interior no dominante
- Tiempo de instrucción no constante
- Consideraciones del sistema (e.g. Mem. virtual)
- Dependencia en los valores de la entrada
- Múltiples parámetros

Manejando la dependencia en las entradas

- Modelos de la entrada imprecisos: Hacer análisis más detallados
- Estimar tiempos de peor caso: Esto provee una garantía de como se comportará el algoritmo
- Utilizar algoritmos randomizados, por ejemplo randomizar los elementos del vector de entrada
- Realizar análisis amortizado

Análisis de peor caso

- Ejemplo: Las operaciones de las estructuras Bag, Stack, Queue implementadas con una lista enlazada requieren tiempo constante en el peor caso.

- Ejemplo: El algoritmo ThreeSum requiere un tiempo total en el orden de N^3 en el peor caso.

Ejemplo: Búsqueda secuencial

- Dado un objeto x, determinar en que posición de un arreglo se encuentra.

```
int buscar(T[] datos, T item) {
    for(int i=0; i<datos.length; i++)
        if (item.equals(datos[i]))
            return i;
    return -1;
}
```

Modelo de costo:
La comparación

- Peor caso: No se encuentra, hace N comparaciones.
- Mejor caso: Aparece en la 1ª posición, hace 1 comparación.

Alta variabilidad entre casos, incluso para el mismo tamaño de la entrada

Ejemplo: Búsqueda secuencial
Análisis de caso medio

Análisis amortizado

- Si una operación costosa se ejecuta con poca frecuencia, se puede *amortizar* su costo sobre el tiempo total requerido.
- Ejemplo: La pila implementada con arreglo y con la operación `resize`(). Se hace una secuencia de N operaciones push. Suponer que el arreglo inicial es de tamaño 4 y se hace M invocaciones del método `add`.

Ejemplo: ResizingArrayBag.java

Ejemplo: Método add con resize

- Se asume $M=2^k$ invocaciones, tamaño inicial 4:

$$
\begin{aligned}
\#accesos &= 1+1+1+1+(8+1)+1+1+1+(16+1)+1+1+1+1+1+1+1+(32+1)+\ldots \\
&= 2^k + \sum_{i=3}^{k} 2^i \\
&= 2^k + \left(\frac{2^{k+1}-1}{2-1} - 2^0 - 2^1 - 2^2 \right) \\
&= 3M - 8
\end{aligned}
$$

$$
\overline{\#accesos} = \frac{3M-8}{M} \sim 3 \quad \text{que es } O(1)
$$

Ejercicio: ArrayList

- La clase `ArrayList` ofrece una implementación de “arreglos de tamaño variable” utilizando la estrategia de hacer `resize` de un arreglo estándar
- Cómo son los tiempos de las operaciones más típicas?
  - `add(E e)`
  - `get(int i)`
  - `remove(int i)`

Análisis de memoria

- Es muy dependiente de la arquitectura del hardware y del lenguaje de programación.
- Se asumen los siguientes modelos (Java, arquitectura de 64bits)

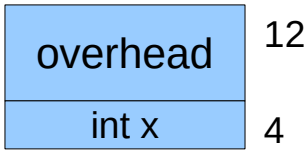
Tipo primitivo	Memoria (bytes)
boolean	1
byte	1
char	2
int	4
long	8
float	4
double	8
referencia	8

Representación de objetos

- Un objeto requiere 12 bytes de overhead (referencia al objeto class, información para el garbage collector, datos de sincronización)
- Luego del objeto se guardan las variables de instancia
- Finalmente se agrega un padding para que el total sea múltiplo de 8 bytes (= 64bits)

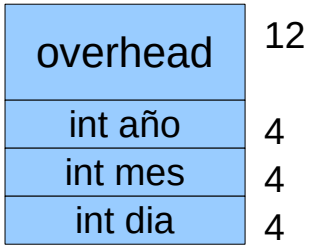
Ejemplos

Integer



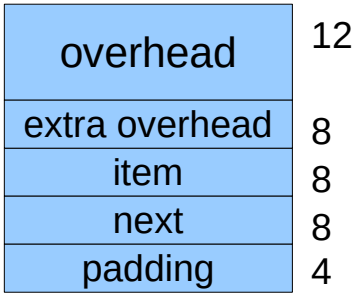
tamaño = 16

Date



tamaño = 24

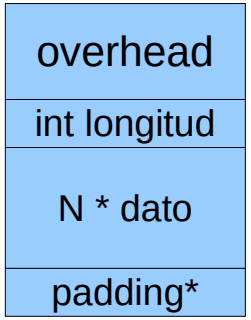
Nodo



tamaño = 40

Arreglos

Arreglo
(tipo primitivo)



12

4

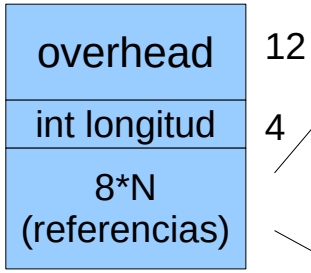
N * tamaño dato

4

tamaño = 16 + N(tamaño tipo)

Instancias

Arreglo
(tipo objeto)



12

4



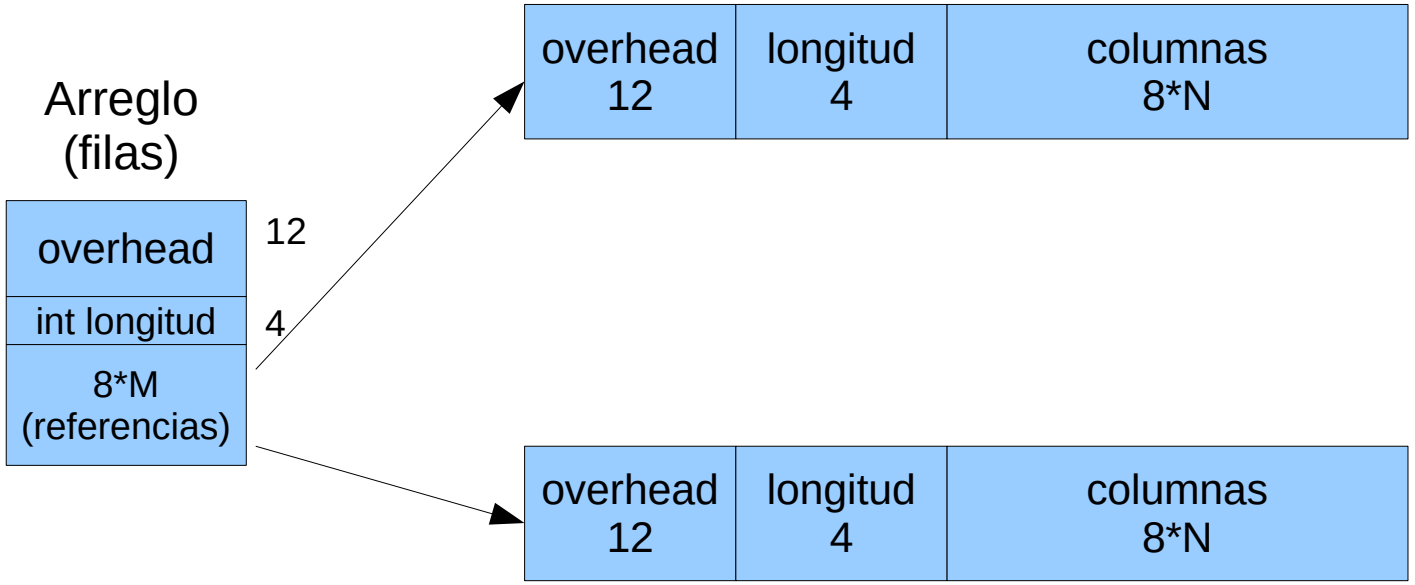


tamaño = 16 + 8*N + N(tamaño instancia)

Arreglos bidimensionales

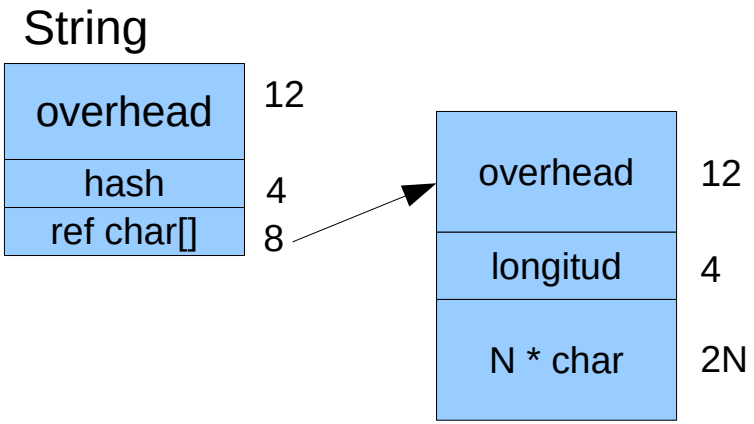
Ejemplo:

```
double[][] matriz = new double[M][N];
```



tamaño total = 16 + 8M + M(16+8N)

Strings (Java 7 y superior)



Total = 24 + 16 + 2*N = 40 + 2*N